Ahojte,

zhrnutie k zadaniu 5. Máte zapísať 3 algoritmy pre prácu s údajmi, z toho 2 algoritmy vyhľadávacích stromov a jeden algoritmus hašovacej tabuľky. Podmienkou je implementovať aspoň 2 algoritmy, nie je dovolené vytvoriť rôzne implementácie jedného algoritmu/stromu, taktiež nie je dovolené vybrať si kombináciu 2,3 a 2,3,4 ako algoritmov. Zároveň musíte vytvoriť testovací program, ktorý nakŕmi všetky vaše implementácie rovnakými údajmi v danom behu a zhodnotí časovú náročnosť jednotlivých typov operácii, ktoré boli vykonávané. Údaje si načítajte vopred do operačnej pamäte či už z STDIN alebo súboru ale **nečítajte ich za behu testovaných algoritmov**, môže to ovplyvniť meranie. Pre porovnanie výkonnosti algoritmov je zaujímavé merať priamo operácie pracujúce s údajmi, teda insert, search, remove. **NEMERAJTE beh celej implementácie,** viď nižšie. Výstupom je koľko času zabrali jednotlivé typy operácii, teda všetky insert operácie zabrali 2000 výpočtových cyklov, všetky search operácie zabrali et cetera. Keďže merať výkonnosť programu iným programom nie je triviálna úloha, najmä ak potrebujete merať len určitú čas kódu odporúčam presunúť meranie času/výkonnosti priamo do implementácie algoritmov. Opäť dávam do pozornosti, že sa máte zamerať aj na vplyv typu údajov, resp. vykonávaných operácii to znamená, že softvér pre matriku vysoko pravdepodobne nebude často vykonávať operáciu remove, občasne insert ale najmä operáciu search. Naproti tomu softvér pre sledovanie zásielok bude takmer rovnako často vykonávať insert aj remove. Preto je potrebné zhodnotiť algoritmy aj z pohľadu vstupných údajov/operácii. Inými slovami niektorý z algoritmov je vhodnejší pre určitý typ použitia, napríklad ak algoritmus dokáže veľmi rýchlo vykonávať operáciu search ale je veľmi pomalý pre operácie insert či remove pravdepodobne nebude najvhodnejší pre použitie kde sa relatívne často vykonávajú práve tieto operácie oproti search. Aj toto je potrebné zhodnotiť v dokumentácii. Taktiež odporúčam jednotlivé meria vykonať opakovane pre rovnaké vstupné údaje, kvôli tomu ako funguje multitasking v operačnom systéme.

Určite nerobte niečo v štýle

```c
void tester(){
    start = clock();
    redBlackTree();
    stop = clock();
}
```

```c
void tester(){
    cpu_ticks += redBlackTree_insert();
}

int redBlackTree_insert(){
    clock_t start = clock();
    char input[40];
    scanf("%s", &input);
    //...
}
```

Ako príklad si môžete zobrať tento pseudokód

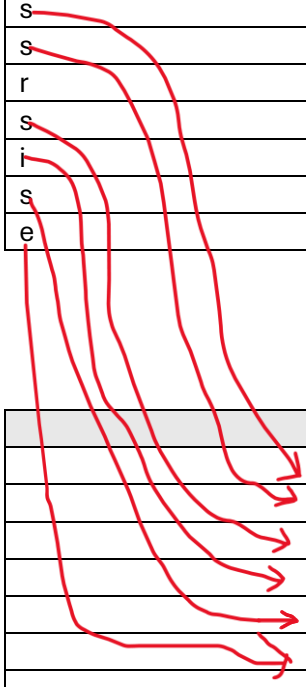
```c
struct cpuCycles
{
    int insert;
    int search;
    int remove;
};
```

```
int tester()
{
    struct cpuCycles redBlackTree;
    char *input = malloc(/*...*/);
    while/for (/*Načítanie celého vstupu do operačnej pamäte*/)
        {
            scanf("%s", input);
            /*Spracovanie vstupu*/
        }

    while/for (/*Beh algoritmu*/)
        {
            switch (/*...*/)
            {
            case 'i':
                // Až tu začína meranie času, respektíve vnútri funkcie,
ktorá vykonáva operáciu, napríklad tak, že funkcia operácie navracia čas
                redBlackTree.insert += redBlackTree_insert(data);
            case 's':
                redBlackTree.search += redBlackTree_search(data);
            case 'r':
                redBlackTree.remove += redBlackTree_remove(data);
            }
        }
}
```

Pre overovanie správnosti funkčnosti bude pre mňa ale aj Vás jednoduchšie mať jednotný formát vstupov a výstupov.

OP[char1/int]	ID[char7/uint]	Krstné[char20]	Priezvisko[char20]
8			
i	1234567	Dominika	Nová
i	7654321	Ján	Kováč
s	1234567		
s	7654321		
r	7654321		
s	7654321		
i	1234567		
s	5432145		
e			



	ID[char7/uint]	Krstné[char20]	Priezvisko[char20]
	1234567	Dominika	Nová
	7654321	Ján	Kováč
	ERROR		
	ERROR		
	ERROR		
	/*Výpis merania*/		

OP- operácia **i**nsert, **s**earch, **r**emove, end. Vstup začína počtom operácii, ktoré sa majú vykonať. Následne vždy bude nasledovať len jeden znak.
ID- unikátne ID, overujete či počet použiteľných cifier(bez vodiacich núl) nie je viac ako 7, na výstupe je vždy sedem
napr. 0000500, overujete či je ID unikátne ak nie je ERROR, ak nejestvuje pri search tiež ERROR. Ak by ste sa rozhodli overovať či je ID menšie ako 10000000, resp =<9999999, upozorňujem, že valídny vstup je aj 0001234567.